YOUR UVM REPO · ADDER_RAL

ADDER → ADDER_RAL: What Changes for a Register Map

Every file compared with ADDER · the DUT's new registers · the register-bus agent · register model, adapter, predictor · build / connect / run phase by phase · results

1. What this folder is: ADDER + a register map (RAL)

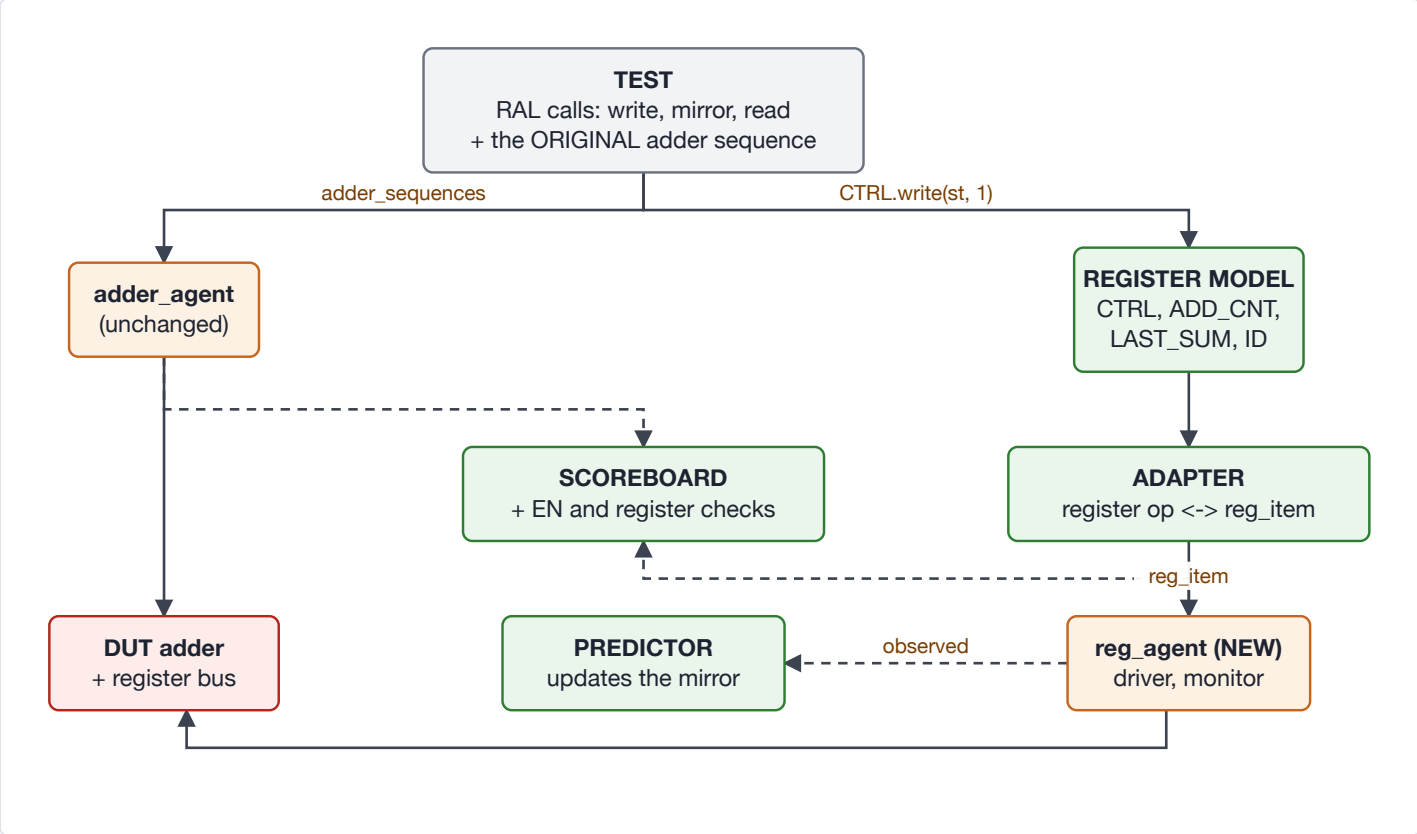
`ADDER_RAL` is your ADDER testbench with **one new idea added**: the test programs and checks the DUT's registers through a **UVM register model (RAL)** — by name (`CTRL.write`), not by address. Everything else is ADDER. Every added or changed line in the code is tagged `// [REG]` (needed because the DUT now has registers) or `// [RAL]` (needed for the register model).

Every file compared with ADDER:

File	Status	Why
`adder_item.sv` , `adder_sequences.sv` , `adder_sequencer.sv` , `adder_driver.sv` , `adder_monitor.sv` , `adder_agent.sv` , `adder_agent_cfg.sv` , `adder_coverage.sv`	**unchanged**	the adder side of the testbench doesn't care that registers exist
`adder.sv`	changed (+37 −3)	register bus + 4 registers; adds only when EN = 1
`adder_if.sv`	changed (+3 −1)	`a_latency` only applies while enabled
`adder_scoreboard.sv`	changed (+34 −2)	third input (register bus): tracks EN, checks register reads
`adder_env.sv`	changed (+16 −1)	register agent, register model, adapter, predictor, their connections
`adder_tests.sv`	changed (+23 −6)	new test `adder_ral_test` : RAL calls around the original traffic
`adder_pkg.sv`	changed (+7)	includes the new files

File	Status	Why
`testbench.sv`	changed (+13 –2)	second interface `reg_if`, wired and published
`reg_if.sv`, `reg_item.sv`, `reg_sequencer.sv`, `reg_driver.sv`, `reg_monitor.sv`, `reg_agent.sv`	**new** `[REG]`	the register-bus agent
`adder_reg_block.sv`, `reg_adapter.sv`	**new** `[RAL]`	the register model and the adapter

How the pieces talk to each other:



2. Step 0 — the DUT needs registers (same change in both folders)

A register map needs **registers**, and the plain adder has none. So the adder gets a tiny **register bus** and four registers. The adding logic itself is unchanged except for one word: it only adds when **EN** is 1.

Address	Register	Access	Meaning
0x0	`CTRL`	RW	bit 0 = **EN**: 1 = add, 0 = ignore inputs (reset 0 — **the test must switch it on**)
0x4	`ADD_CNT`	RO	how many additions were done (hardware counts by itself)
0x8	`LAST_SUM`	RO	the last sum
0xC	`ID`	RO	always `0xADD0` — a constant, handy to check the bus works

The bus (one access at a time): the testbench raises `reg_valid` with `reg_wr`, `reg_addr`, `reg_wdata` and holds them; the DUT answers with a one-cycle `reg_ready`, and for a read puts the value on `reg_rdata` in that same cycle.

What changed in `adder.sv` (lines tagged `// [REG]`):

```
// new ports
input  logic reg_valid, reg_wr;  input logic [3:0] reg_addr;  input logic [31:0] reg_wdata;
output logic reg_ready;          output logic [31:0] reg_rdata;

// the ONLY change to the adding logic: "&& en"
out_valid <= in_valid && en;
if (in_valid && en) begin sum <= sum_next; carry <= sum_next[N]; end

// new: the register block
reg_ready <= reg_valid && !reg_ready;                       // answer one cycle after the request
if (reg_valid && !reg_ready) begin
  if (reg_wr && reg_addr == 4'h0) en <= reg_wdata[0];       // write CTRL
  case (reg_addr)                                           // read any register
    4'h0: reg_rdata <= {31'b0, en};
    4'h4: reg_rdata <= {16'b0, add_cnt};
    4'h8: reg_rdata <= 32'(sum);
    4'hC: reg_rdata <= 32'h0000_ADD0;
  endcase
end
if (out_valid) add_cnt <= add_cnt + 1;                      // count every result
```

A lesson that came out of the run: ADDER's assertion `a_latency` said "every input produces an output next cycle" (`in_valid |=> out_valid`). With an enable bit that is **no longer true** — with EN = 0 the DUT must ignore the input. The virtual-sequence test in `ADDER_VIRTUAL_SEQ` (which disables the adder on purpose) made it fail. **When you add a feature, old checks may need updating**: `adder_if.sv` now reads `in_valid && en |=> out_valid`, and `tb_top` connects the DUT's `en` into the interface (`assign vif.en = dut.en;`) so the assertion can see it.

3. Step 1 — a register-bus agent (same in both folders)

The register bus is a **second interface**, so it gets its own agent — the same shape as `adder_agent`, with its own item, driver, monitor and sequencer.

New file	What it does	Same idea as in ADDER
`reg_if.sv`	wires + clocking blocks + 2 assertions (request held until ready; ready is a 1-cycle pulse)	`adder_if.sv`
`reg_item.sv`	one access: `wr`, `addr`, `data` (read data filled in by the driver)	`adder_item.sv`
`reg_sequencer.sv`	sequencer for register items	`adder_sequencer.sv`
`reg_driver.sv`	drives one access, holds `reg_valid` until `reg_valid && reg_ready`, copies `reg_rdata` into the item for reads	`adder_driver.sv`
`reg_monitor.sv`	one `reg_item` per completed access → analysis port	`adder_monitor.sv`

New file	What it does	Same idea as in ADDER
`reg_agent.sv`	builds monitor (+ driver, sequencer when active), connects them	`adder_agent.sv`

Scoreboard changes (`adder_scoreboard.sv`, tagged `[REG]`) — it now has a **third** input:

```
`uvm_analysis_imp_decl(_reg)                                    // third write function: write_reg()
uvm_analysis_imp_reg #(reg_item, adder_scoreboard) imp_reg;
bit en;                                                         // what the scoreboard believes
CTRL.EN is

function void write_in(adder_item txn);
  if (!en) begin n_ignored++; return; end                       // disabled: expect NO result
  ...
function void write_reg(reg_item t);
  if (t.wr) begin if (t.addr == 4'h0) en = t.data[0]; return; end   // writes tell us the mode
  case (t.addr)                                                      // reads are checked
    4'h4: check_reg("ADD_CNT",  t.data, n_out);
    4'h8: check_reg("LAST_SUM", t.data, last_sum);
    4'hC: check_reg("ID",       t.data, 32'h0000_ADD0);
  endcase
endfunction
```

Other small changes: `testbench.sv` creates `reg_if`, connects its six wires to the DUT and puts it in `config_db` for `uvm_test_top.env.reg_ag.*`; in `adder_env.sv` the config is now published to `"*"` instead of `"ag"`, because the new `reg_ag` needs it too.

4. Step 2 — the register model (`adder_reg_block.sv`, new)

The model is a **description** of the 4 registers: name, width, access policy, reset value, address. No pins, no bus code.

```
class ctrl_reg extends uvm_reg;
  rand uvm_reg_field EN;
  virtual function void build();
    EN = uvm_reg_field::type_id::create("EN");
    EN.configure(this, 1, 0, "RW", 0, 1'b0, 1, 1, 0);       // 1 bit at bit 0, RW, reset 0
  endfunction
endclass
// add_cnt_reg: CNT 16 bits "RO", volatile = 1 (hardware counts by itself)
// last_sum_reg: SUM N+1 bits "RO", volatile;   id_reg: 32 bits "RO", reset 0xADD0

class adder_reg_block extends uvm_reg_block;
  virtual function void build();
    default_map = create_map("default_map", 'h0, 4, UVM_LITTLE_ENDIAN);
    CTRL = ctrl_reg::type_id::create("CTRL");  CTRL.configure(this);  CTRL.build();
    ...
    default_map.add_reg(CTRL,     'h0, "RW");                // name -> address
    default_map.add_reg(ADD_CNT,  'h4, "RO");
    default_map.add_reg(LAST_SUM, 'h8, "RO");
    default_map.add_reg(ID,       'hC, "RO");
    lock_model();
  endfunction
endclass
```

Why volatile for ADD_CNT and LAST_SUM: the hardware changes them without any register write, so the model **cannot predict** them. Marking them volatile tells UVM not to compare them in `mirror()` and to skip them in the reset test; the **scoreboard** checks their values instead.

5. Step 3 — the adapter (`reg_adapter.sv`, new)

The adapter is the only RAL code that knows the bus: `reg2bus` turns "write CTRL = 1" into a `reg_item {wr=1, addr=0, data=1}`; `bus2reg` turns the completed item back into a result. Change the bus (APB, AXI-Lite…) and only this file and the agent change.

```
function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);
  reg_item t = reg_item::type_id::create("t");
  t.wr = (rw.kind == UVM_WRITE);  t.addr = rw.addr;  t.data = rw.data;
  return t;
endfunction
function void bus2reg(uvm_sequence_item bus_item, ref uvm_reg_bus_op rw);
  reg_item t;  $cast(t, bus_item);
  rw.kind = t.wr ? UVM_WRITE : UVM_READ;  rw.addr = t.addr;  rw.data = t.data;  rw.status = UVM_IS_OK;
endfunction
```

6. Step 4 — the env: build and connect (`adder_env.sv`,

changed)

Phase	New lines	Why
build	`reg_ag = reg_agent::type_id::create("reg_ag", this);`	the register bus needs an agent
build	`regmodel = adder_reg_block::type_id::create("regmodel"); regmodel.build();`	the model is a `uvm_object`: create it, then `build()` makes the registers and the map
build	`adapter = reg_adapter::type_id::create("adapter");` `predictor = uvm_reg_predictor #(reg_item)::type_id::create("predictor", this);`	translator + mirror updater
connect	`regmodel.default_map.set_sequencer(reg_ag.sqr, adapter);`	"register operations go out on this sequencer, through this adapter"
connect	`regmodel.default_map.set_auto_predict(0);` `predictor.map = regmodel.default_map; predictor.adapter = adapter;` `reg_ag.ap.connect(predictor.bus_in);`	the model's mirror follows what the **monitor** really saw
connect	`reg_ag.ap.connect(sb.imp_reg);`	the scoreboard also sees register traffic

7. Step 5 — the test (`adder_tests.sv`, changed)

ADDER's test started one sequence. The new `adder_ral_test` wraps the **same sequence** with register work:

```
rst_seq = uvm_reg_hw_reset_seq::type_id::create("rst_seq");   // 1. built-in: reads every register,
rst_seq.model = env.regmodel;  rst_seq.start(null);           //    compares with its reset value
env.regmodel.CTRL.write(status, 1);                           // 2. enable: one bus write to 0x0
env.regmodel.CTRL.mirror(status, UVM_CHECK);                  //    read back + compare with the model
seq.start(env.ag.sqr);                                        // 3. the ORIGINAL 100 additions
env.regmodel.ADD_CNT.read(status, val);                       // 4. read status; the scoreboard checks
it
env.regmodel.LAST_SUM.read(status, val);
```

Phase by phase — what's new compared with ADDER:

Phase	ADDER	ADDER_RAL adds
build	test → env → adder agent, scoreboard, coverage	+ register agent, register model (`build()`), adapter, predictor
connect	monitor → scoreboard, coverage	+ map → register sequencer + adapter; register monitor → predictor and scoreboard
run	100 additions	reset check → enable via RAL → mirror check → 100 additions → read ADD_CNT, LAST_SUM

Phase	ADDER	ADDER_RAL adds
check / report	leftovers, pass/fail	+ register checks counted

What happens on `CTRL.write(status, 1)`: the map finds address 0x0 → `reg2bus` builds a `reg_item` → it goes through `reg_ag.sqr` → `reg_driver` drives `reg_valid` … `reg_ready` → the DUT sets EN → `reg_monitor` sees the write → the **predictor** updates the mirror ("CTRL = 1") and the **scoreboard** sets its `en`.

8. Results and how to run

Check	Result
Reset test (`uvm_reg_hw_reset_seq`)	CTRL and ID at their reset values (ADD_CNT, LAST_SUM skipped: volatile)
`CTRL.write` + `mirror(UVM_CHECK)`	EN = 1, model and DUT agree
100 additions	100 / 100 correct, 0 assertion failures
`ADD_CNT` read through RAL	**100** — scoreboard: OK; `LAST_SUM` : OK; `ID` : `0xADD0` OK

Run: `./run.sh` → `results/sim.log` , `results/dump.vcd` . Search the log for `[REG]` / `[RAL]` -related lines: `ID read add0 OK` , `ADD_CNT read through RAL = 100` .

Try this to learn: comment out `env.regmodel.CTRL.write(status, 1);` — EN stays 0, the DUT ignores all 100 inputs, and the scoreboard reports `ignored(EN=0)=100` and **Test FAIL**: it saw no results at all. That's why the pass rule demands *outputs > 0* — without it, a test that never enabled the DUT would look like a pass.